

Coalesce Partners Senior Backend Engineer Coding Challenge — Real-Time PnL Engine

Real-Time PnL Engine

Expected time	Approximately 3-4 focused hours. AI coding tools are welcome; we evaluate the final result and the reasoning documented in your README.
Calculation period	2026-08-01 00:00:00 UTC to 2026-08-02 00:00:00 UTC
Reporting currency	USDT
Interface	CLI, simple API, or another reasonable interface. No frontend required.
Deliverables	Source code, tests, and a short README. Git repository or zip archive.

How to read this document. **Part 1 is the exercise** — read it and you can start building. **Part 2 is specification detail**: boundary conditions, dataset assumptions, and edge-case policy. You do not need to absorb Part 2 before you begin; use it as reference while building, and review it before finalizing your submission. Its length reflects our attempt to save you a round of clarifying emails, not the amount of work involved.

Part 1 — The Exercise

Background

Our investment platform consolidates trading and portfolio data across multiple traders, accounts, and exchanges. For this exercise, build a simplified real-time PnL engine that maintains position state and calculates PnL as new trading and market events arrive.

The goal is not to build a fully productionized trading system. We are primarily interested in how you model the problem, structure your code, handle state and edge cases, and reason about how the system could evolve in production.

The Data

You will be provided with four CSV files. The dataset contains multiple traders, accounts, exchanges, and instruments. Trade and funding rows may not be presented in chronological order.

opening_positions.csv

Position snapshot at 2026-08-01 00:00:00 UTC, immediately before any event in the exercise is processed. Positive quantity is long, negative is short.

```
timestamp,trader,venue,venue_account,symbol,quantity,avg_entry_price
2026-08-01T00:00:00Z,TRADER_A,BINANCE,ACCOUNT_A,BTCUSDT,2.0,60000
```

trades.csv

quantity is always positive; side determines direction. fee is always a positive quantity of fee_asset and represents a trading cost. Fees may be charged in USDT or another asset such as BNB.

```
timestamp,venue,trade_id,trader,venue_account,instrument_type,symbol,side,
quantity,price,fee,fee_asset
2026-08-01T01:00:00Z,BINANCE,T00001,TRADER_A,ACCOUNT_A,FUTURES,BTCUSDT,BUY
,0.5,61000,12.20,USDT
2026-08-01T01:30:00Z,BINANCE,T00002,TRADER_A,ACCOUNT_A,FUTURES,BTCUSDT,SEL
L,0.3,62000,0.015,BNB
```

funding.csv

Funding payments on perpetual futures positions. Positive amounts are funding received; negative amounts are funding paid.

```
timestamp,event_id,trader,venue,venue_account,symbol,asset,amount
2026-08-01T04:00:00Z,F0001,TRADER_A,BINANCE,ACCOUNT_A,BTCUSDT,USDT,-150
```

prices.csv

Mark-price updates for traded instruments and BNBUSDT.

```
timestamp,symbol,price
2026-08-01T03:00:00Z,BTCUSDT,62500
2026-08-01T03:00:00Z,BNBUSDT,805
```

All PnL is reported in USDT. A fee charged in a non-USDT asset is converted using the latest available price at or before the trade timestamp. For example, if BNBUSDT is 800 at 10:00 and 805 at 10:05, a 0.01 BNB fee on a trade at 10:03 is valued using the 10:00 price.

What to Calculate

Position state and reporting grain. Maintain position quantity and cost-basis state independently for each (trader, venue, venue_account, symbol). Do not net positions across venue accounts before calculating realized and unrealized PnL. The report itself is grouped by trader and symbol: produce it by summing final quantity and each PnL component across that trader's account-level states.

Realized PnL. Calculated when an existing position is reduced or closed. You may choose an appropriate cost-basis methodology — we intentionally do not prescribe FIFO, weighted-average cost, or anything else. Document which you chose, why, and any assumptions.

Unrealized PnL. Calculated on the remaining open position using the latest available mark price, for both long and short positions. Never use a price later than the valuation timestamp. If no eligible mark price exists, report the value as explicitly unavailable rather than substituting an execution price or zero, and continue producing the other components.

Funding PnL. Sum the funding events associated with each trader and symbol.

Fees. The USDT value of all trading fees. Fees reduce PnL.

Total PnL = Realized PnL + Unrealized PnL + Funding PnL – Fees

If any component required for Total PnL is unavailable, report Total PnL as unavailable rather than treating that component as zero.

Incremental Updates

Beyond the one-off report, the engine should behave like a running system.

- **Initial load.** Event time is authoritative: process events in timestamp order rather than file or ID order. Deduplicate trades and funding events on load.
- **Live processing.** After the initial load, new trades, mark prices, and funding events may arrive. A simple interface such as `engine.process(event)` is sufficient. In-order events should update state incrementally rather than triggering a full recalculation, and an exact repeat of a known event must be a no-op.

- **Late events.** A valid event may arrive with a timestamp earlier than state that has already been processed. Arrival order must not silently change the economic result. At minimum, detect such an event and reject or defer it explicitly; do not process it incorrectly in arrival order. Correct insertion with bounded replay is a stretch goal described in Part 2.

Expected Output

For each trader and symbol, your system should be able to produce output similar to:

TRADER_A

BTCUSDT

Final Quantity	2.20
Realized PnL	+1,250.00
Unrealized PnL	+3,400.00
Funding PnL	-150.00
Fees	35.00

Total PnL	+4,465.00

The exact format is up to you. A CLI, simple API, or another reasonable interface is acceptable.

Deliverables

1. Source code
2. Tests
3. A short README covering: how to run the solution; your overall approach; your chosen cost-basis methodology and reasoning; key assumptions; edge cases handled; important trade-offs; and anything you would improve with more time.

Practical notes. Use whichever mainstream language you are strongest in — `engine.process(event)` is illustrative, not a language requirement. Third-party libraries are fine. No external database, message broker, or container setup is required; in-memory state is sufficient. Please submit a Git repository link or a zip archive including your tests, and check that the run instructions work from a clean checkout.

If You Are Short On Time

Prioritize in this order:

1. **Correct end-of-period PnL.** Deduplication, chronological processing, multi-asset fee conversion, and realized, unrealized, and funding PnL for long and short positions, reported per trader and symbol.
2. **Incremental in-order processing** through `engine.process(event)`, with an exact duplicate event handled as a no-op and a late event explicitly rejected or deferred rather than misapplied.
3. **As-of views, interim missing-price behaviour, and bounded replay of late events.**

A submission that is strong on items 1 and 2, with item 3 addressed in the README as documented assumptions and design notes, is a valid submission. Please prioritize correctness, clarity, and reasoning over completeness.

Production Design

Assume the system eventually grows to 100 traders, 10 exchanges, 50,000 open positions, and several million events per day. In your README, briefly address:

- What would you change first, and what would you deliberately keep simple?
- How would you manage and persist state, and rebuild it after a failure?
- How would you reconcile calculated positions and PnL against exchange data?

Bullet points are fine — half a page is plenty, and we will go deeper on this in the interview rather than expecting a written design document. We are more interested in your trade-offs and reasoning than in

naming particular technologies.

What We Evaluate

- correctness of position and PnL calculations
- code structure and readability
- data modeling
- testing approach
- incremental state management
- handling of multi-asset fee conversion
- awareness of real-world data issues
- engineering trade-offs
- production and scaling judgment

There is no single expected architecture or implementation.

Part 2 — Specification Detail

Reference material. Look things up here as they come up, and review this part before finalizing your submission — it contains rules that affect correctness.

Dataset Assumptions

- `instrument_type` is FUTURES for every row in this dataset. It is provided for realism only; you do not need to include it in your state key.
- `prices.csv` is venue-agnostic. There is one global mark-price series per symbol, shared across all venues.
- The funding asset is USDT for every row. If you choose to support non-USDT funding, apply the same conversion rule as for fees.
- `trade_id` is unique per venue, not globally. Treat `(venue, trade_id)` as the identity of a trade, and `(venue, event_id)` as the identity of a funding event.
- Opening positions supply only an average entry price. If you use a lot-based cost-basis methodology, treat each opening position as a single lot at `avg_entry_price`.
- The files may contain rows outside the calculation window. Exclude any trade or funding event with a timestamp before the period start or at or after the period end.

Calculation Window and Boundaries

- Trade and funding events are treated as a half-open interval: `[2026-08-01 00:00:00 UTC, 2026-08-02 00:00:00 UTC)`.
- Price rows are reference data and are not subject to that window in the same way. A price before the period start is eligible if it is the latest price at or before the relevant timestamp, and the tick at exactly `2026-08-02 00:00:00 UTC` is eligible as the closing mark.
- For all price lookups, "at or before" is inclusive: a price with the same timestamp as the trade or valuation must be preferred over an earlier price.

Precision and How Reference Outputs Are Compared

- Use a numeric representation appropriate for financial calculations. Do not round intermediate quantities, fee conversions, cost basis, or PnL components. Round only displayed results.
- **Total PnL, Funding PnL, and Fees** are compared against reference outputs within ± 0.01 USDT per (trader, symbol).
- **Final quantity** is compared separately, in instrument units, within $1e-8$.

- **Realized PnL and Unrealized PnL** are assessed individually for internal consistency rather than against a fixed reference. Each must follow the methodology you declare in your README, and together with funding and fees they must reconcile to your reported total.

Fee Conversion

For every non-USDT fee in the supplied dataset, an eligible conversion price exists at or before the trade. For live events where no eligible conversion price exists, apply the same explicit-unavailable policy described below for missing marks; do not silently substitute a later price or treat the fee as zero.

Idempotency and Conflicting Corrections

- An exact repeat of a known event must be a no-op.
- If an event arrives with a known identity but different content, this is a conflicting correction rather than a duplicate. Choose a policy — for example reject and surface, or upsert and recompute — and document it. Do not apply it as if it were a new independent event.
- Index prices by symbol and timestamp. Treat (symbol, timestamp) as the natural key for a price, and document how a conflicting correction to an existing key is handled.

As-of Valuation and Missing Prices

The end-of-period requirement — never valuing at a future price, and reporting an unavailable mark explicitly — is covered in Part 1. Beyond that:

- **[Stretch]** The engine should be able to produce an as-of view after any processed event, not only the end-of-period report. A mark price lookup must use the latest price at or before the requested as-of timestamp.
- **[Stretch]** Where unrealized PnL is undefined, expose that state explicitly — for example null, an unavailable marker, or a typed error — rather than reporting zero. Document the policy you chose.
- **[Stretch]** If you implement this section, include at least one test covering a position whose first mark price arrives after trading has begun.

Late and Out-of-Order Events

Minimum behaviour [Required]. Detect that an event predates already-processed state for its partition and handle it explicitly — reject it with a clear error, or defer it — rather than applying it in arrival order as though it were current. A late price must not be dropped silently; if you cache derived valuations, a late price for a symbol must at minimum invalidate the cached values it would have changed.

Bounded replay [Stretch]. If you go further:

- **Late trade.** Insert it into the affected account-level event stream and replay only the affected (trader, venue, venue_account, symbol) partition, from a preceding checkpoint or from its opening position. Do not recalculate unrelated traders, accounts, or symbols.
- **Late funding.** Can be incorporated after idempotency checks, because its amount is already supplied.
- **Late or corrected price.** Must invalidate derived valuations that used a different latest-at-or-before price, including relevant non-USDT fee conversions. Recompute only the affected symbol and time range.
- **Scope.** An ordered in-memory event history per state partition is sufficient. Durable checkpoints and streaming infrastructure are not required.
- **Tests.** Include at least one test for a late trade whose insertion changes the subsequent cost basis.

Data and Edge Cases

Real-world trading data is not always perfectly clean or ordered. The supplied dataset may contain cases such as:

- multiple traders and accounts
- long and short opening positions
- position increases and reductions
- positions crossing through zero
- fees paid in different assets
- duplicate events
- events that are not presented in chronological order
- multiple price updates for the same symbol

You are not expected to build production-grade handling for every possible failure mode within the time limit. Please make reasonable decisions and document important assumptions. We are particularly interested in how you think about idempotency, event ordering, missing prices, position state, data validation, and recoverability.